Cloud-Native Trade Anomaly Detection & Reporting Platform

Rapport technique

Professeur : Abdennadher N.

Auteurs : Binjos Adnan
Falcinelli Théo
Schmid Alexandre

GitHub : https://github.com/AdnanBinBin/dnext-trade-platform

Version : 1.0

03 novembre 2026

MSE | MASTER OF SCIENCE IN ENGINEERING

1 Contents

1 INTRODUCTION 4

1.1 OBJECTIFS DU PROJET 4
1.2 STRUCTURE DU RAPPORT 4

2 ANALYSE DES EXIGENCES 5

2.1 DONNÉES ET DÉTECTION DES ANOMALIES 5
2.2 EXIGENCES FONCTIONNELLES 6
2.3 EXIGENCES NON FONCTIONNELLES ET CONTRAINTES 6
2.4 CONTRAINTES DU PROJET 6

3 CONCEPTION ET ARCHITECTURE 8

3.1 ARCHITECTURE GÉNÉRALE 8
3.2 STOCKAGE ET TRAITEMENT ANALYTIQUE DES DONNÉES 8
3.2.1 Solutions envisagées 9
3.2.2 Comparaison avec les exigences du projet 9
3.2.3 Choix retenu 10
3.3 BACKEND ET TRAITEMENT 10
3.3.1 Solutions envisagées 11
3.3.2 Comparaison avec les exigences du projet 11
3.3.3 Choix retenu 12
3.4 FRONTEND 13
3.4.1 Solutions envisagées 13
3.4.2 Comparaison avec les exigences du projet 14
3.4.3 Choix retenu 14
3.5 INTELLIGENCE ARTIFICIELLE 15
3.5.1 Solutions envisagées 15
3.5.2 Comparaison avec les exigences du projet 16
3.5.3 Choix retenu 16
3.6 ARCHITECTURE RETENUE 17

4 IMPLÉMENTATION 19

4.1 INGESTION DES DONNÉES 19
4.2 TRAITEMENT ANALYTIQUE ET BACKEND 19
4.2.1 Implémentation du mirror matching 19
4.2.2 Stockage des résultats dans la couche Gold 20
4.2.3 Implémentation de l'API backend 21
4.2.4 Déploiement et intégration AWS 22
4.3 FRONTEND ..... **ERROR! BOOKMARK NOT DEFINED.**

5 TESTS ET ÉVALUATION 24

5.1 VALIDATION DES EXIGENCES FONCTIONNELLES 24
5.2 TESTS ET RÉSULTATS OBTENUS 24
5.2.1 Validation des données et du mirror matching 24
5.2.2 Tests unitaires et d'intégration 25
5.3 LIMITES DE LA SOLUTION 26
5.3.1 Actualisation des données 26
5.3.2 Performances et scalabilité 26
5.3.3 Qualité des données et détection des anomalies 27
5.3.4 Limites des explications générées par IA 27
5.3.5 Sécurité et exploitation 28
5.4 PERSPECTIVES D'ÉVOLUTION POUR DNEXT 28

6 CONCLUSION.....29

7 TABLES DES FIGURES.....30

8 BIBLIOGRAPHY.....31

1 Introduction

Les données du commerce international peuvent être utilisées pour analyser les échanges de marchandises entre différents pays. Toutefois, pour un même échange, les valeurs déclarées par le pays exportateur et le pays importateur peuvent être très différentes. Bien sûr, une partie de cette différence reste normale, notamment car les exportations sont généralement déclarées selon une valeur FOB (*Free On Board*), tandis que les importations utilisent une valeur CIF (*Cost, Insurance, and Freight*) qui inclut des coûts supplémentaires liés au transport et à l'assurance.

Dans ce contexte, DNEXT souhaite analyser la fiabilité de données commerciales en comparant les déclarations d'importation et d'exportation correspondant à un même échange. Cette méthode, appelée « mirror matching », permet notamment d'identifier des écarts anormaux entre les valeurs déclarées par les deux partenaires commerciaux.

Ce projet vise donc à concevoir et implémenter une plateforme cloud permettant d'ingérer et traiter des données provenant de UN Comtrade, de détecter ces anomalies et de les visualiser au travers d'une interface web. De plus, une composante d'intelligence artificielle (IA) générative doit également permettre de fournir des éléments de contexte pouvant aider à expliquer les anomalies détectées.

1.1 Objectifs du projet

Les principaux objectifs du projet peuvent être résumés de la manière suivante :

1. Concevoir une architecture cloud adaptée aux besoins du projet, et justifier les principaux choix technologiques
2. Ingérer et stocker le jeu de données UN Comtrade fourni dans une solution adaptée à son analyse
3. Implémenter le *mirror matching* et les règles permettant de détecter les anomalies entre les déclarations d'importation et d'exportation.
4. Exposer les résultats au travers d'une API.
5. Développer une interface web permettant de configurer l'analyse et de visualiser les anomalies sous la forme d'une *heatmap*.
6. Intégrer une composante IA générative permettant d'apporter des éléments d'explication aux anomalies détectées.
7. Tester et valider le fonctionnement des différents composants de la plateforme.

1.2 Structure du rapport

Ce rapport est organisé selon les différentes étapes du projet :

- Le chapitre 2 : présente les exigences, les données utilisées ainsi que le fonctionnement de la détection d'anomalies
- Le chapitre 3 : présente ensuite la conception de l'architecture, et les choix technologiques effectués
- Le chapitre 4 : décrit l'implémentation des différents composants de la plateforme
- Le chapitre 5 : présente les tests, les résultats obtenus et les principales limites observées et présente les possibles améliorations futures
- Le chapitre 6 : conclut le rapport

2 Analyse des exigences

Cette section présente les données utilisées dans le cadre du projet, le principe de détection des anomalies, ainsi que les principales exigences fonctionnelles (FR) et non fonctionnelles (NFR) de la plateforme.

2.1 Données et détection des anomalies

Le jeu de données fourni provient de UN Comtrade et contient 200'587 enregistrements d^'échanges commerciaux bilatéraux mensuels à partir de janvier 2019. Il couvre 10 pays exportateurs, leur partenaires commerciaux ainsi que 7 catégories de produits agricoles.

Les données sont organisées selon les champs présentés dans le tableau suivant.

Champ	Description
YearMonth	Mois de la déclaration
Reporter	Pays ayant effectué la déclaration
Flow	Type de flux (Importation / Exportation)
Partner	Pays partenaire
HS4	Code HS4 du produit
Product	Nom du produit
Value_USD	Valeur déclarée en USD
Qty_t	Quantité nette en tonnes

La valeur d'une exportation est exprimée en FOB (*Free On Board*), tandis que celle d'une importation est exprimée en CIF (*Cost, Insurance and Freight)*.

La détection des anomalies repose donc sur le principe de *mirror matching* mentionné précédemment. Pour un mois et un produit données, une exportation déclarée d'un pays A vers un pays B est mise en correspondance avec l'importation déclarée par le pays B depuis le pays A. Ces 2 lignes représentent donc les deux déclarations d'un seul même flux commercial et ne doivent donc pas être additionnées.

Le prix unitaire de chaque déclaration est calculé à partir de la valeur et de la quantité écahngée

$$Prix\ unitaire = \frac{Value_USD}{QTY_t}$$

L'écart entre les deux déclarations peut ensuite être calculé selon la formule suivante :

$$\text{Écart}\ (\%) = \frac{CIF - FOB}{FOB} \times\ 100$$

Une paire est considérée comme anormale lorsque le prix FOB est supérieur au prix CIF, ou également dans le cas ou l'écart entre les deux dépasse 35%. Dans les autre cas, les deux déclarations sont considérées comme cohérentes.

Il est également possible qu'une seule des deux déclarations soit présente dans le jeu de données. Dans ce cas, l'enregistrement reste valide, mais le *mirror matching* ne peut pas être effectué pour ce flux.

2.2 Exigences fonctionnelles

Les exigences fonctionnelles (FR, *Functional Requirements)* décrivent les principales fonctionnalités que la plateforme doit fournir. Elles sont principalement issues du cahier des charges du projet.

ID	Exigence fonctionnelle
FR-01	Le système doit permettre d'ingérer le jeu de données UN Comtrade fourni.
FR-02	Le système doit mettre en correspondance les déclarations d'importation et d'exportation correspondantes.
FR-03	Le système doit calculer les prix unitaires FOB et CIF et détecter les anomalies selon les règles définies.
FR-04	Les résultats des traitements doivent être accessibles au travers d'une API documentée.
FR-05	L'utilisateur doit pouvoir sélectionner un pays exportateur, un produit et une fréquence temporelle.
FR-06	Le système doit afficher les résultats sous forme de heatmap pour les dix principaux partenaires commerciaux.
FR-07	Les anomalies doivent être clairement identifiables et sélectionnables depuis la heatmap.
FR-08	La sélection d'une anomalie doit permettre d'obtenir une explication générée par IA à partir des informations disponibles

2.3 Exigences non fonctionnelles et contraintes

Les exigences non fonctionnelles (NFR, *Non functional Requirements*) décrivent quant à elles les propriétés attendues de la solution au-delà de ses fonctionnalités. Elles concernent notamment l'architecture, la scalabilité, le déploiement et les contraintes liées au projet.

ID	Exigence non fonctionnelle	Critère de validation
NFR-01	L'architecture doit permettre une évolution du volume de données et de la charge de traitement.	Analyse de la scalabilité de l'architecture proposée.
NFR-02	Les principaux composants de la plateforme doivent être découplés afin de pouvoir évoluer indépendamment.	Séparation claire entre les composants et leurs interfaces.
NFR-03	L'interface web doit pouvoir être déployée indépendamment du backend.	Frontend et backend déployés séparément.
NFR-04	La solution doit rester compatible avec le budget AWS disponible pour le projet.	Vérification des services et ressources AWS utilisés.
NFR-05	Les principaux choix technologiques doivent être justifiés en fonction des besoins du projet.	Comparaison et justification des alternatives lors de la conception de l'architecture.
NFR-06	Le fonctionnement des principaux composants de la plateforme doit pouvoir être testé et validé.	Mise en place de tests couvrant les principaux composants et flux de la plateforme.

2.4 Contraintes du projet

En complément des exigences fonctionnelles et non fonctionnelles définies dans les sections précédentes, plusieurs éléments liés au contexte du projet influencent aussi les choix techniques et architecturaux :

- Environnement AWS : DNEXT travaille déjà avec l'écosystème AWS. Les services de cette plateforme sont donc privilégiés afin de rester cohérent avec l'environnement du client et d'éviter la multiplication des fournisseurs cloud.
- Périmètre des données : le projet se concentre sur le jeu de données UN Comtrade fourni qui constitue la source de données principales de la plateforme.

- Temps disponible : le projet doit être réalisé entre le 6 octobre et le 3 novembre 2026. Ce délai est relativement court et doit être pris en compte dans le choix des technologies et limite l'intérêt de solutions nécessitant une mise en œuvre ou une administration importante.
- Complexité proportionnée : compte tenu du volume de données, du périmètre et du temps disponible, ce point regroupe ces contraintes et défini que l'architecture doit rester proportionnée aux besoins. L'ajout de composants supplémentaires doit donc apporter un bénéfice concret au projet.

3 Conception et architecture

Cette section présente la conception de la plateforme, les principales alternatives technologiques qui ont été considérées ainsi que les choix finaux retenus pour répondre au mieux aux exigences définies dans le chapitre 2.

3.1 Architecture générale

L'architecture proposée repose sur une séparation entre la préaration et l'analyse des données, les services applicatifs et l'interface utilisateur. Cette organisation permet de distinguer les traitements analytiques, exécutés de manière relativement ponctuelle, des fonctionnalités accessibles aux utilisateurs à travers l'application web.

La plateforme est organisée autour de 4 blocs principaux :

- Stockage et traitement des données :
  - Les données issues d'UN Comtrade sont importées, validées et préparées avant d'être analysées. *Le mirror matchin*g permet ensuite de calculer les écarts entre les déclarations d'importation et d'exportation et d'identifier les anomalies. Les résultats sont précalculés et conservés afin d'éviter de répéter ces traitements à chaque consultation.
- Backend et API :
  - Le backend assure la communication entre l'interface utilisateur et les services de la plateforme. Il permet notamment de récupérer les anomalies selon les filtres sélectionnées et de demander la génération de rapports
- Frontend :
  - L'interface Web permet aux utilisateurs d'explorer les données, de visualiser les anomalies sous forme de *heatmap* et de sélectionner les cas nécessitant une analyse complémentaire. Elle reste indépendante des traitements analytiques.
- Génération des rapports IA :
  - Lorsqu'un utilisateur sélectionne une anomalie, un processus distinct rassemble les informations pertinentes, sollicite un modèle d'IA générative pour produire une explication contextualisée, puis génère et conserve un rapport consultable ultérieurement.

Cette architecture cherche à privilégier des composants managés et une exécution « à la demande » afin de limiter les ressources actives en permanence. Elle vise également à conserver une séparation claire entre les différentes responsabilités des différents blocs, tout en restant proportionnée au volume de données et au temps disponible pour le projet.

Les sections suivantes présentent les solutions envisagées pour chacun de ces blocs leur comparaison avec les exigences du projet, ainsi que les raisons ayant motivé les choix technologiques retenus.

3.2 Stockage et traitement analytique des données

Les données utilisées par la plateforme sont principalement destinées à des opérations de lecture et d'analyse. Le jeu de données fourni contient 200'587 observations mensuelles issues d'UN Comtrade, couvrant 10 pays exportateurs et 7 catégories de produits agricoles.

Les traitements nécessitent notamment de filtrer ces données, d'associer les déclarations d'importation et d'exportation correspondantes et finalement de claculer les écarts entre prix unitaires FOB et CIF.

Plusieurs plateformes cloud proposent des services adaptés au stockage et à l'analyse de ce type de données, notamment Google BigQuery ou encore Snowflake. Ces solutions pourraient techniquement répondre aux besoins techniques du projet, mais DNEXT utilise déjà l'écosystème AWS, ce qui constitue donc un élément important du contexte du projet. Afin de limiter la multiplication des fournisseurs et de conserver une architecture cohérente avec l'environnement en place du client, l'étude détaillée se concentre donc sur les services disponibles dans l'écosystème AWS.

3.2.1 Solutions envisagées

Si on se concentre donc sur AWS, plusieurs solutions peuvent répondre à ce besoin. Les principales alternatives envisagées sont récapitulées dans le tableau suivant [1] [2] [3] [4]:

Solution	Avantages	Inconvénients
RDS / PostgreSQL	- Solution relationnelle simple - Support SQL - Jointures adaptées au *mirror matching* - Accès simple depuis un backend	- Base de données active à maintenir - Plus orientée vers des usages relationnels et transactionnels que vers un stockage analytique
Redshift Serverless	- Data warehouse conçu pour les traitements analytique - SQL adapté à des volumes importants - Capacité gérée automatiquement	- Fonctionnalitées peu justifiées par le faible volume du projet - Complexité d'implémentation peu justifiée par le volume du projet
S3 + Glue + Athena	- Stockage objet simple - Services managés et serverless - Requêtes SQL - Adapté aux données analytiques peu modifiées - Stockage et calcul découplés	- Plusieurs services à configurer - Moins adapté à des données nécessitant de nombreuses modifications transactionnelles
Architecture hybride	- Permet de séparer le stockage analytique du stockage utilisé pour servir les résultats à l'application	- Ajoute un second système de stockage et augmente la complexité de l'architecture

Les différentes solutions sont techniquement capables de traiter le volume actuel. Le choix ne repose donc pas principalement sur leurs performances maximales, mais sur leur adéquation aux exigences et à l'échelle du projet

3.2.2 Comparaison avec les exigences du projet

Afin de faciliter la comparaison, le code couleur suivant a été utilisé :

- Vert : indique qu'une solution répond directement à l'exigence
- Jaune : indique qu'une solution répond à l'exigence avec certaines réserves ou un compromis
- Orange : indique qu'une solution est peu adaptée au besoin considéré.

Exigence / contrainte	RDS / PostgreSQL	Redshift Serverless	S3 + Glue + Athena	Architecture hybride
FR-01: Ingestion et stockage des données	Adapté	Adapté	Adapté	Adapté
FR-02 / FR-03: Mirror matching et calcul des anomalies	SQL et jointures	SQL analytique	SQL avec Athena	Selon les composants
FR-04: Accès aux résultats depuis le backend	Accès direct	Accès direct	Via Athena	Plusieurs systèmes
NFR-01: Évolution du volume et de la charge	Scaling de la base	Adapté aux grands volumes	Stockage et calcul découplés	Très flexible
NFR-04:Compatibilité avec budget AWS	Ressources provisionnées	Peu justifié pour ce volume	Paiement à l'usage	Plusieurs services

Complexité proportionnée au projet	Faible	Plus importante	Modérée	Complexité supplémentaire
Bilan	Simple et suffisante	Puissante mais surdimensionnée	**Approche retenue**	Peu justifiée actuellement

Cette comparaison montre notamment que RDS/PostgreSQL constitue une solution parfaitement suffisante pour le volume actuel.

Redshift de son côté répond également au besoin et offre davantage de capacités analytiques, mais celles-ci sont peu justifiées pour un jeu de données de la taille de celui utilisé pour le projet

L'architecture hybride apporte quant à elle une séparation supplémentaire entre stockage analytique et stockage applicatif, au prix d'une complexité dont le projet n'a pas nécessairement besoin.

3.2.3 Choix retenu

La solution retenue pour cette partie de l'architecture est d'utilisé Amazon S3, associé à AWS Glue Data Catalog et Amazon Athena.

En effet, bien que RDS/PostgreSQL soit tout à fait suffisant pour traiter le volume actuel de données, la solution S3+Glue+Athena correspond davantage à la nature analytique des données qui sont traitées et à leur fréquence de modification limitée. Cette solution permet de conserver les données dans S3 et de les interroger en SQL avec Athena, sans maintenir en plus une base de données active en permanence. Le fonctionnement à l'usage est également intéressant pour ce projet car les traitements et consultations restent assez ponctuels.

De l'autre côté Redshift Serverless offre certes des capacités analytiques plus importantes, mais celles-ci ne sont clairement pas justifiable pour les quelque 200'000 observations du jeu de données. Quant à l'architecture hybride, elle ajouterait un second système de stockage sans apporter de bénéfice suffisamment important pour justifier cette complexité supplémentaire.

De plus, il a été décidé de définir le stockage S3 en trois zones logiques :

- *Bronze / Raw : conservation des fichiers Excel originaux.*
- *Silver / Curated : données validées, nettoyées et converties au format Parquet.*
- *Gold / Results : conservation des résultats précalculés du mirror matching et de la détection des anomalies.*

AWS Glue Data Caatalog de son côté permet de référencer les données et leurs schémas, tandis qu'Athena fournira les fonctionnalités SQL nécessaires à leur analyse. De plus, le format Parquet sera utiliisé pour faciliter les lectures analytiques et limiter le volume de données parcourues à chaque requête.

Cette organisation permet aussi de séparer les données sources, les données préparées et les résultats des traitements, et en cas d'ajout de nouvelles données, le pipeline pourra être relancé sans modifier le fonctionnement du *frontend*.

3.3 Backend et traitement

Si on se concentre maintenant sur le *backend*, ce dernier doit assurer l'interface entre l'application web, et les traitements de la plateforme. C'est-à-dire permettre au *frontend* de récupérer les résultats nécessaires à l'affichage de la *heatmap* à travers une API. Il doit aussi permettre l'exécution du traitement de mirror matching lors de la mise à jour des données.

Dans une approche ou les résultats de *mirror matching* sont précalculés. Les requêtes effectuées par les utilisateurs ne nécessitent donc pas de recalculer l'ensemble des correspondances entre

importations et exportations. Le *backend peut* simplement interroger les résultats déjà produits, tandis que le traitement analytique n'est exécuté que lorsque les données sources sont mises à jour.

3.3.1 Solutions envisagées

Il existe plusieurs services AWS qui permettraient d'exécuter le backend et les traitements de la plateforme. Trois approches principales ont été mises en avant [5] [6] [7] [8]:

- AWS Lambda
- ECS/Fargate (application conteneurisée)
- EKS (orchestration Kubernetes)

De plus, Athena mentionné au chapitre 3.2.1 constitue aussi une possibilité pour exécuter directement une partie du traitement analytique en SQL sur les données stockées dans S3.

Approche	Avantages	Inconvénients
AWS Lambda	- Pas de serveur à administrer - Exécution à la demande - Scaling automatique - Paiement à l'usage - Faible charge d'administration	- Durée d'exécution limitée pour les fonctions standards - Environnement d'exécution plus contraint qu'un serveur /conteneur classique - Moins adapté à des traitements continus / longs
ECS + Fargate	- Permet d'exécuter une application dans des conteneurs Docker - Pas de serveurs EC2 à provisionner ou maintenir avec Fargate - Contrôle des ressources CPU et mémoire du conteneur - Adapté aux API, applications web et traitements de données - Plus flexible que Lambda pour les traitements plus long	- Nécessite de construire et gérer les images des conteneurs - Configuration ECS, réseau et tâches plus importante que pour Lambda - Plus de composants à déployer
EKS / Kubernetes	- Orchestration avancée des applications conteneurisées (replicas, scaling, …) - Gestion du déploiement et de la mise à l'échelle de plusieurs workloads - Adapté aux architectures comprenant de nombreux services	- Nécessite une plus grande expertise et temps investi pour comprendre et configurer Kubernetes - Introduit un cluster, et une couche d'orchestration supplémentaires (plus complexe)
Athena pour le traitement analytique	- Exécution de requêtes SQL directement sur les données S3 - Service serverless, sans infrastructure de calcul à administrer - Permet d'effectuer des jointures et transformations SQL - Possibilité de matérialiser le résultat d'une requête dans une nouvelle table S3	- Ne constitue pas un serveur applicatif et ne remplace donc pas le backend/API - Approche adaptée uniquement si le mirror matching peut être exprimé correctement en SQL

3.3.2 Comparaison avec les exigences du projet

Le backend et le traitement analytique remplissent en soit deux rôles distincts. Tout d'abord, le premier consiste à exposer les fonctionnalités de la plateforme au frontend à travers une API, tandis que le second consiste à effectuer les tâches de *mirror matching* et produire les résultats montrant les anomalies.

Les solutions ont donc été comparées de manière séparée pour ces deux besoins. Le code couleur reprend celui de la section précédente :

- Vert : indique qu'une solution répond directement à l'exigence
- Jaune : indique qu'une solution répond à l'exigence avec certaines réserves ou un compromis
- Orange : indique qu'une solution est peu adaptée au besoin considéré.

Hébergement du backend et de l'API

Exigence	AWS Lambda	ECS+Fargate	EKS/Kubernetes
FR-04 : API documentée	Adapté à une API légère	Adapté	Adapté
FR-05 à FR-07 : Requêtes venant du frontend	Exécution à la demande	Service disponible en continu	Adapté
NFR-01 : Evolution de la charge	Scaling automatique	Scaling des tâches	Scaling avancé
NFR-02 : Composants découplés	Fonctions indépendantes	Services conteneurisés	Services Kubernetes
NFR-04 : Budget AWS	Paiement à l'usage	Ressources de tâches à financer	Infrastructure plus importante nécessaire
Contrainte de temps et de complexité	Faible administration	Configuration supplémentaire	Complexité importante
Bilan	Très adapté	Très flexible	Surdimensionné pour le projet

Exécution du mirror matching

Approche	Traitement applicatif	Traitement SQL avec Athena
FR-02 : Mirror matching	Jointures / correspondances sont implémentées dans le code	Jointures SQL entre déclarations d'import et d'export
FR-03 : Calcul des anomalies	Formules et règles aussi implémentées dans le code	Calculs FOB/CIF/GAP et classification en SQL
Intégration avec S3	Lecture des données à développer	Lecture directe des tables référencées par Glue
Précalcul des résultats	Exécution du traitement puis écriture des résultats	Création et alimentation d'une table Gold avec Athena SQL
NFR-04 : Budget AWS	Dépend du service exécutant le traitement	Facturation à l'usage des requêtes
Complexité proportionnée	Logique de calcul à développer et environnement à gérer	Réutilisation du service analytique déjà retenu
Bilan	Flexible, mais nécessite un traitement applicatif supplémentaire	Cohérent avec S3 + Glue + Athena

Cette comparaison montre que les différentes solutions envisagées sont capables de répondre aux besoins du *backend*. Toutefois, compte tenu du volume limité de données, de la fréquence ponctuelles des traitements, et du temps disponible pour le projet, les solutions nécessitant une infrastructure plus importante sont à nouveau difficiles à justifier.

De ce fait, AWS Lambda apparait ainsi comme une solution adaptée pour exposer les fonctionnalités de la plateforme à travers une API, sans maintenir un service applicatif actif en permanence.

Concernant le *mirror matching*, les opérations nécessaires reposent principalement sur des jointures et des calculs pouvant être exprimés en SQL. L'utilisation d'Athena permettrait donc de réutiliser le moteur analytique déjà envisagé pour le stockage des données, sans introduire un composant de calcul supplémentaire.

3.3.3 Choix retenu

Au final, la solution retenue pour le backend sera d'utilisé Amazon API Gateway et AWS Lambda. En effet, API Gateway expose les endpoints http de la plateforme, tandis que Lambda exécute la logique applicative nécessaire au traitement des requêtes.

Ce choix a surtout été motivé par la faible charge attendue et le caractère ponctuel des requêtes. Contrairement à une application conteneurisée fonctionnant en continu, Lambda permet ici une exécution à la demande et aussi limite ainsi les ressources à maintenir. ECS/Fargate aurait aussi pu répondre au besoin, mais aurait nécessité davantage de configuration. EKS a été écarté en raison de sa complexité, peu justifiée pour une application de cette taille.

Le *mirror matching* est réalisé à l'aide de requêtes SQL exécutées par Athena sur les données préparées dans la couche Silver. Les résultats sont ensuite conservés dans la couche Gold de S3 et peuvent être consultés par le backend sans relancer le traitement analytique.

Pour le MVP, ce calcul est exécuté après l'ingestion initiale du jeu de données fourni, à l'aide d'un script de déploiement dédié. Le traitement reste donc indépendant des consultations effectuées par les utilisateurs.

En cas d'ajout de nouvelles données, le calcul peut être relancé afin de reconstruire la couche Gold. Son déclenchement automatique après chaque ingestion n'est toutefois pas encore implémenté et constitue une évolution possible de la plateforme.

3.4 Frontend

L'interface utilisateur de son côté constitue le point d'accès principal à la plateforme. Elle doit permettre de sélectionner les paramètres d'analyse, de consulter les résultats du *mirror matching* sous la forme d'une *heatmap* et finalement de sélectionner une anomalie afin d'obtenir des informations complémentaires. Le frontend communique avec le backend à travers une API et ne réalise pas directement les traitements analytiques.

3.4.1 Solutions envisagées

L'interface web peut être compilée en fichiers statiques HTML, CSS et JavaScript. Son hébergement ne nécessite donc pas de serveur applicatif exécuté en permanence.

Dans ce contexte, deux solutions AWS ont principalement été envisagées, à savoir [9] [10] :

- Amazon S3 associé à CloudFront
- AWS Amplify Hosting

Leurs avantages et inconvénients sont présentés dans le tableau suivant :

Solution	Avantages	Inconvénients
S3 + CloudFront	- Hébergement de fichiers statiques dans uns erveur applicatif - Contrôle de la configuration du stockage et de la distribution - Distribution du contenu via un CDN	- Configuration de plusieurs services AWS - Gestion des autorisations d'accès entre S3 et CloudFront - Automatisation du déploiement à mettre en place séparément
AWS Amplify Hosting	- Hébergement managé des applications web - Intégration avec un dépôt Git - Compilation et déploiement automatisés - Gestion simplifiée de l'hébergement HTTPS	- Moins de contrôle direct sur l'infrastructure sous-jacente - Dépendance aux mécanismes de déploiement d'Amplify

En soit, les deux solutions permettent d'héberger une application web statique et de la render accessible aux utilisateurs. La principale difference concerne le niveau de configuration necessaire ainsi que la gestion des déploiements.

3.4.2 Comparaison avec les exigences du projet

Les deux solutions présentées dans la section précédente répondent aux besoins fonctionnels de l'interface. Leur comparaison repose donc principalement sur les aspects liés à l'hébergement, au niveau déploiement et à la complexité de la mise en œuvre.

Le code couleur utilisé reprend celui des sections précédentes, à savoir :

- Vert : La solution répond directement à l'exigence
- Jaune : La solution répond à l'exigence avec certaines réserves
- Orange : la solution est peu adaptée au besoin considéré

Exigence / Contrainte	S3 + CloudFront	AWS Amplify Hosting
FR-05 : Sélection des paramètres	Adapté	Adapté
FR-06 / FR-07 : Heatmap interactive	Adapté	Adapté
FR-08 : Interaction avec l'IA via l'API	Adapté	Adapté
NFR-03 Déploiement indépendant du backend	Adapté	Adapté et automatisable
NFR-04 : Budget AWS	Facturation selon les ressources utilisées	Facturation à l'usage de l'hébergement et des builds
Contraintes de temps et de complexité	Configuration de plusieurs services	Configuration et déploiement simplifiés
Bilan	Flexible, mais potentiellement plus de configurations	Adapté aux besoins et aux contraintes de temps du projet

On remarque que les deux solutions sont techniquement adaptées aux besoins de la plateforme. Les fonctionnalités de l'interface dépendent principalement de l'application développée et ne constituent donc pas un critère déterminant pour le choix de l'hébergement.

S3 + CloudFront offre davantage de contrôle sur la configuration de l'infrastructure, tandis qu'Amplify simplifie la compilation et le déploiement de l'application.

Compte tenu du temps disponible et de la volonté de limiter la complexité opérationnelle, cette simplicité constitue un avantage important pour le projet.

3.4.3 Choix retenu

La solution retenue pour l'hébergement du frontend est AWS Amplify Hosting.

En effet, les 2 approches étudiées permettent de répondre aux besoins fonctionnels de l'interface. Toutefois, Amplify simplifie le processus de déploiement en prenant en charge la compilation et la publication de l'application à partir d'un dépôt Git.

S3 + CloudFront reste également une solution adaptée, mais aurait nécessité davantage de configuration pour obtenir un processus de déploiement comparable. Compte tenu du temps disponible pour le projet, cette complexité supplémentaire n'apporte pas de bénéfice important ici.

Par ailleurs, l'interface sera développée avec Vue.js (due à l'expérience préalable de certains membres internes au projet) et déployée indépendamment du backend. Elle communiquera avec les fonctions Lambda à travers API Gateway, ce qui permet de faire évoluer l'interface sans modifier directement les traitements analytiques.

3.5 Intelligence artificielle

La composante d'intelligence artificielle vise à compléter la détection des anomalies par la génération d'un rapport explicatif. Contrairement au *mirror matching*, qui repose sur des règles déterministes, cette étape doit permettre d'interpréter les résultats obtenus et de présenter les éléments pertinents dans un format compréhensible pour un analyste.

La génération d'un rapport nécessite plusieurs opérations :

- Récupérer les données de l'anomalie sélectionnée
- Rassembler des informations historiques
- Solliciter un modèle d'IA generative
- Puis finalement produire et conserver le document final.

Ces opérations peuvent nécessiter plusieurs appels à des services externes et présenter des durées d'exécution variables. Il est donc préférable que leur traitement soit indépendant des requêtes de consultation de la heatmap.

3.5.1 Solutions envisagées

Plusieurs approches peuvent être envisagées pour intégrer un modèle d'IA générative à la plateforme. Les principales alternatives sont présentées dans le tableau suivant. [11] [12] [13]

Solution	Avantages	Inconvénients
API d'un fournisseur d'IA externe	Intégration relativement simple ; accès à des modèles performants ; infrastructure du modèle entièrement gérée	Dépendance à un fournisseur supplémentaire ; gestion des accès et de la transmission des données en dehors de l'environnement AWS
Amazon Bedrock	Accès à différents modèles de fondation ; service managé ; intégration avec IAM et les autres services AWS ; facturation à l'usage	Coût dépendant du nombre de requêtes et de tokens ; dépendance à la disponibilité des modèles selon les régions
Hébergement d'un modèle sur une infrastructure dédiée	Contrôle important sur le modèle et son environnement d'exécution ; personnalisation possible	Déploiement et maintenance plus complexes ; ressources de calcul à dimensionner ; coûts potentiellement peu justifiés pour une utilisation ponctuelle

Au-delà du choix du modèle, deux approches principales sont possibles pour organiser la génération des rapports.

- La première consiste à effectuer toutes les opérations dans une même fonction Lambda, de manière synchrone. Cette solution est relativement simple à implémenter, mais oblige le client à attendre la fin du traitement et rend la gestion des erreurs plus difficile.
- La seconde consiste à utiliser un workflow asynchrone, dans lequel la demande est enregistrée puis traitée en plusieurs étapes indépendantes. AWS Step Functions constitue une solution adaptée à cette approche, puisqu'il permet d'orchestrer plusieurs fonctions Lambda et de définir des mécanismes de reprise en cas d'erreur.

Cette seconde approche ajoute un service à configurer, mais présente un intérêt particulier pour la génération de rapports, dont la durée peut varier selon les appels au modèle d'IA et les traitements nécessaires à la production du document.

3.5.2 Comparaison avec les exigences du projet

Les différentes approches permettent de répondre au besoin de génération d'explications défini par l'exigence FR-08. Leur comparaison repose donc principalement sur leur intégration à l'architecture AWS, leur complexité de mise en œuvre et leur capacité à gérer des traitements de durée variable.

Exigence / contrainte	API d'IA externe	Amazon Bedrock	Modèle hébergé
FR-08 : génération d'explications	Adapté	Adapté	Adapté
NFR-01 : évolution de la charge	Capacité gérée par le fournisseur, sous réserve des quotas	Service managé, soumis à des quotas	Dimensionnement à assurer
NFR-02 : intégration aux composants AWS	Intégration par API	Intégration native avec IAM et Lambda	Intégration supplémentaire nécessaire
NFR-04 : maîtrise des coûts	Facturation à l'usage	Facturation à l'usage	Ressources de calcul à financer
Complexité de mise en œuvre	Faible à modérée	Modérée	Plus importante
Bilan	Solution simple, mais ajoute un fournisseur	Approche retenue	Peu justifiée pour le MVP

Amazon Bedrock a été privilégié, car il permet d'utiliser des modèles génératifs sans devoir administrer leur infrastructure d'exécution, tout en restant dans l'écosystème AWS déjà retenu pour les autres composants.

Concernant l'orchestration, une exécution synchrone serait suffisante pour une démonstration très simple, mais elle est moins adaptée à la génération d'un document complet. L'utilisation de Step Functions permet de découpler cette opération de l'API et de mieux gérer les étapes successives.

Ce choix répond notamment à l'objectif de séparation des composants (NFR-02), tout en conservant une architecture dont les ressources de calcul sont principalement exécutées à la demande.

3.5.3 Choix retenu

La solution retenue repose sur Amazon Bedrock pour la génération du contenu, associé à AWS Step Functions et plusieurs fonctions Lambda pour l'orchestration des traitements. Le modèle choisi quant à lui est Claude Haiku 4.5, accessible à travers Amazon Bedrock au moyen d'un profil d'inférence européen. Ce choix permet de bénéficier d'un modèle de génération de texte sans nécessiter de déploiement ni de maintenance de l'infrastructure du modèle.

Lorsqu'un utilisateur demande un rapport, l'API enregistre la demande et déclenche un workflow Step Functions. Celui-ci exécute successivement la collecte du contexte depuis Athena, la rédaction de l'explication par Bedrock et la génération du document PDF.

Cette approche permet de retourner rapidement une confirmation au frontend, puis de consulter l'état d'avancement du traitement. Step Functions permet également de gérer les nouvelles tentatives lors de certaines erreurs temporaires, notamment lorsque le service Bedrock ne peut pas traiter immédiatement une requête. Les rapports générés sont conservés dans Amazon S3, tandis qu'Amazon DynamoDB stocke les informations relatives aux demandes et à leur état d'exécution. Cette séparation permet de retrouver un rapport déjà généré sans nécessairement relancer le modèle d'IA.

Enfin, la génération du PDF est réalisée par du code applicatif plutôt que directement par le modèle. Les valeurs numériques et les graphiques proviennent des données analytiques, tandis que l'IA est

principalement utilisée pour produire l'explication narrative. Cette distinction permet de conserver davantage de contrôle sur les informations présentées dans le document.

Pour le MVP, les explications reposent sur les résultats du mirror matching et sur les données historiques disponibles dans la couche Gold. L'intégration de sources externes complémentaires n'a pas été réalisée et constitue une possibilité d'évolution qui sera discutée au chapitre 5.

3.6 Architecture retenue

Les choix technologiques présentés dans les sections précédentes conduisent à l'architecture AWS illustrée à la Figure de la page suivante.

La plateforme repose donc sur un pipeline de préparation des données, suivi d'un traitement analytique réalisé avec Athena. Les résultats du *mirror matching* sont conservés dans la couche Gold, puis exposés à travers une API Gateway et des fonctions Lambda.

L'interface web, hébergée indépendamment avec AWS Amplify, permet de consulter les anomalies et de demander la génération de rapports. Cette dernière fonctionnalité est prise en charge par un workflow asynchrone utilisant Step Functions, Bedrock et des services de stockage dédiés.

Cette organisation permet de séparer les traitements analytiques des consultations utilisateur et des opérations de génération de rapports, tout en privilégiant des services managés et une exécution à la demande.

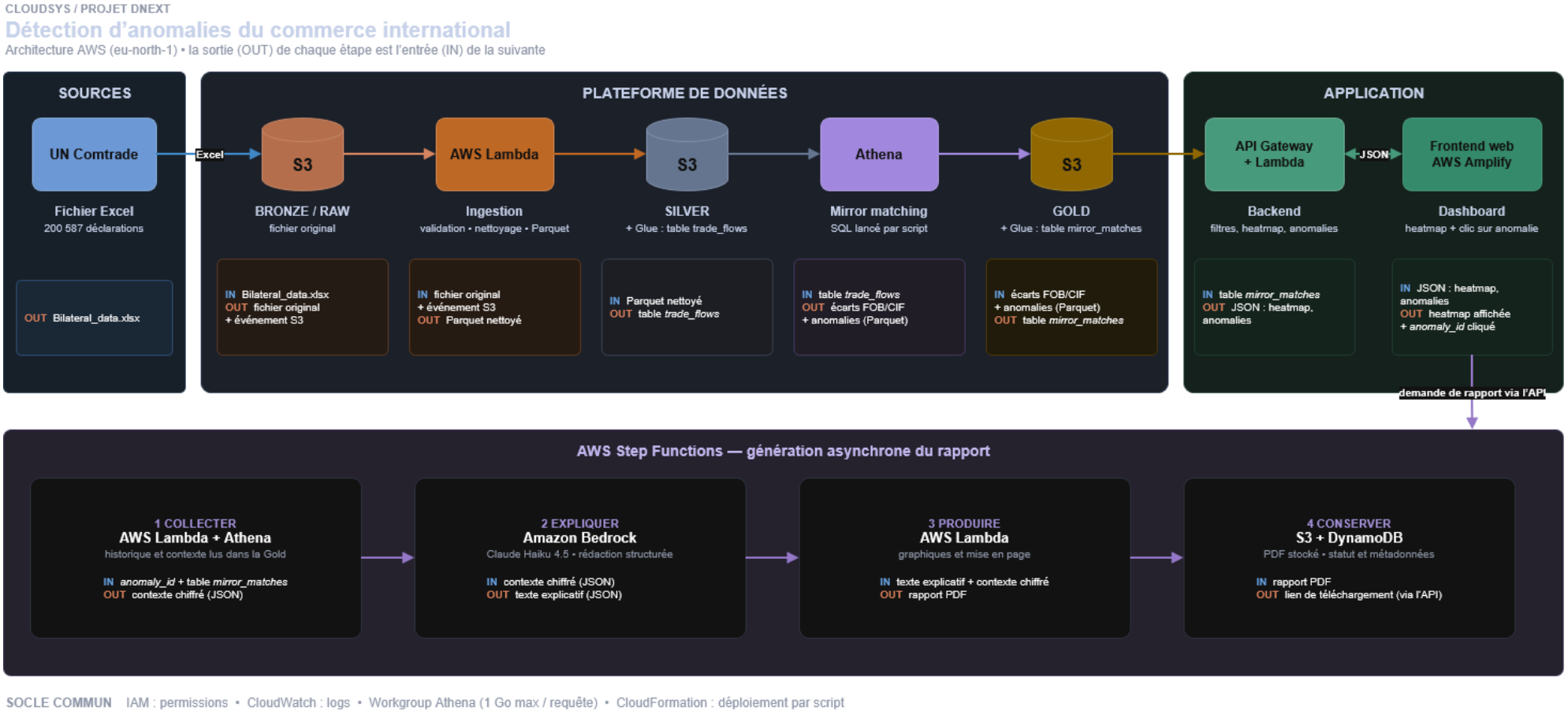


Figure 1: Architecture générale AWS retenue

4 Implémentation

Cette section décrit l'implémentation réalisée sur la base des décisions d'architectures présentées dans la section précédente.

4.1 Ingestion des données

L'ensemble de l'infrastructure de cette partie est décrit dans un template AWS CloudFormation et déployé par un script (`deploy.sh`). Il est ainsi possible de recréer l'environnement à l'identique en une seule commande, notamment après une suppression des ressources du compte.

Le fichier Excel UN Comtrade est déposé tel quel dans un bucket S3 Bronze/Raw versionné, sous le préfixe `comtrade/`. Ce dépôt déclenche automatiquement une fonction AWS Lambda (Python, pandas) qui vérifie la structure du fichier puis valide chaque ligne : période, flux, codes pays et produits, périmètre des dix exportateurs, valeurs et doublons. Les lignes invalides sont écartées dans une table de quarantaine avec leur motif de rejet. Les lignes suspectes mais légitimes, par exemple un tonnage manquant ou nul, sont conservées et marquées par des indicateurs de qualité. Les codes sont normalisés et quelques colonnes descriptives sont ajoutées : exportateur et importateur du flux, base FOB/CIF, traçabilité vers la ligne source. En revanche, aucun calcul métier n'est effectué à ce stade.

Les données sont ensuite écrites au format Parquet, partitionnées par année, dans un bucket S3 Curated, accompagnées d'un rapport d'ingestion. Elles sont référencées dans le Glue Data Catalog (table `trade_flows`) et interrogeables en SQL via Athena. Sur le jeu de données fourni, les 200 587 lignes sont intégrées sans rejet. Ce résultat est vérifié par des tests unitaires et par un test de bout en bout exécuté sur Athena.

4.2 Traitement analytique et backend

L'implémentation du backend repose sur la séparation entre le calcul des anomalies et leur consultation depuis l'application web, conformément aux choix présentés à la section 3.3.

Le premier traitement consiste à exploiter les données préparées par le pipeline d'ingestion afin d'effectuer le *mirror matching* et d'identifier les écarts anormaux entre les déclarations d'importation et d'exportation. Les résultats sont ensuite conservés dans une nouvelle zone de stockage afin de pouvoir être consultés sans relancer l'ensemble du traitement analytique.

Le second composant constitue le backend applicatif. Il expose les résultats précalculés à travers une API HTTP, en prenant en charge les paramètres de filtrage, l'agrégation des données nécessaires à la *heatmap* et la consultation des anomalies individuelles.

4.2.1 Implémentation du mirror matching

Tout d'abord, le calcul du *mirror matching* a été implémenté sous la forme de requêtes SQL exécutées avec Amazon Athena. Il s'appuie directement sur la table `dnext_trade.trade_flows`, produite par le pipeline d'ingestion et référencée dans AWS Glue Data Catalog.

Cette table contient notamment les pays exportateur et importateur normalisés, la période, le code produit HS4, le type de déclaration, la valeur commerciale et la quantité échangée. Ces informations permettent de retrouver les deux déclarations correspondant à un même flux commercial.

Une jointure SQL de type `FULL OUTER JOIN` est utilisée afin d'associer les déclarations d'exportation (FOB) et d'importation (CIF) selon quatre critères qui sont

- La période
- Le pays exportateur
- Le pays importateur
- Le code produit HS4.

L'utilisation d'une jointure complète permet également de conserver les déclarations pour lesquelles aucune correspondance n'existe. En effet, une déclaration sans miroir ne doit pas être considérée comme cohérente, mais comme un cas pour lequel la comparaison des prix ne peut pas être effectuée.

Pour chaque paire valide, les prix unitaires FOB et CIF sont calculés en divisant la valeur déclarée par la quantité correspondante. L'écart relatif entre ces deux prix est ensuite déterminé selon la formule présentée à la section 2.1.

Les anomalies quant à elles sont identifiées à partir des deux règles définies dans le cahier des charges à savoir, un prix CIF inférieur au prix FOB, ou un écart positif supérieur à 35 %.

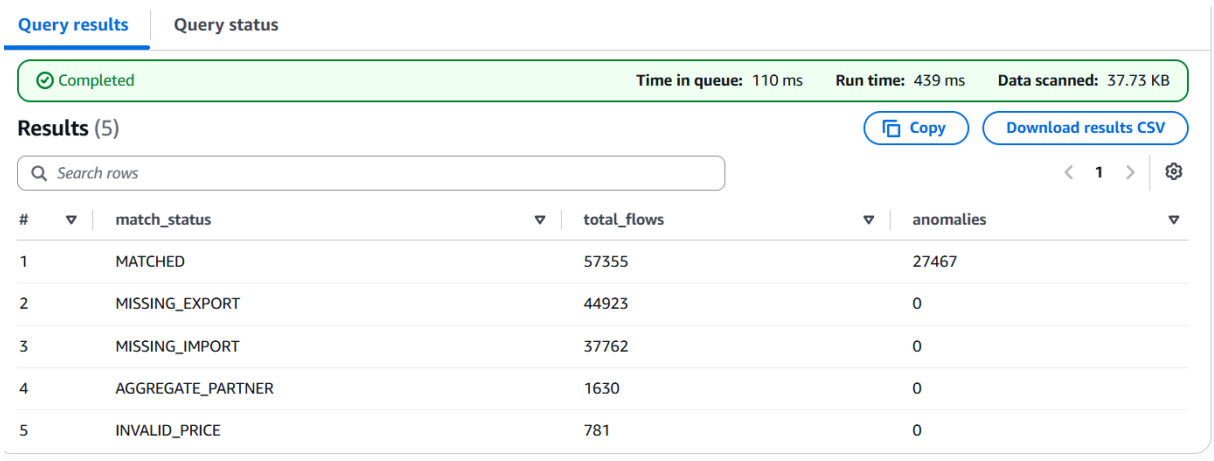
En complément, le traitement attribue un statut à chaque flux afin de distinguer les correspondances valides, les déclarations sans miroir, les prix non calculables et les partenaires commerciaux agrégés. Cette distinction permet notamment d'éviter que des données incomplètes soient interprétées comme des échanges cohérents.

4.2.2 Stockage des résultats dans la couche Gold

Afin d'éviter de répéter le *mirror matching* à chaque consultation du frontend, les résultats du traitement sont précalculés puis conservés dans une zone S3 dédiée, appelée Gold, correspondant à la zone *Results* prévue dans l'architecture.

Un bucket Amazon S3 distinct a été créé à cet effet. Les résultats sont stockés au format Parquet et référencés dans le Glue Data Catalog à travers la table `dnext_trade.mirror_matches`. Cette table contient notamment les informations permettant d'identifier chaque flux commercial, les valeurs et quantités déclarées, les prix unitaires FOB et CIF, l'écart calculé ainsi que le statut de correspondance et le motif éventuel d'anomalie. Un identifiant déterministe (anomaly_id) est également généré à partir des caractéristiques du flux commercial. Il permet de retrouver individuellement une observation depuis l'API et pourra notamment être utilisé lors de la demande d'un rapport explicatif.

L'exécution du traitement sur le jeu de données fourni a permis de produire 142'451 flux commerciaux distincts, dont 57'355 correspondances valides entre déclarations d'importation et d'exportation. Parmi ces dernières, 27'467 ont été identifiées comme anormales selon les règles définies précédemment. Comme l'illustre la Figure ci-dessous, les autres observations sont conservées et classifiées selon leur situation, notamment lorsqu'une déclaration miroir est absente, qu'un prix ne peut pas être calculé ou que le partenaire correspond à une zone agrégée. Cette classification permet de distinguer les anomalies effectivement détectées des cas pour lesquels la comparaison n'est pas possible.


Query results | Query status

Completed — Time in queue: 110 ms — Run time: 439 ms — Data scanned: 37.73 KB

Results (5) — Copy — Download results CSV

Search rows — 1

#	match_status	total_flows	anomalies
1	MATCHED	57355	27467
2	MISSING_EXPORT	44923	0
3	MISSING_IMPORT	37762	0
4	AGGREGATE_PARTNER	1630	0
5	INVALID_PRICE	781	0

Figure 2: Répartition des flux commerciaux selon leur statut de correspondance et nombre d'anomalies détectées

Pour le MVP, le traitement Gold a été exécuté après l'ingestion initiale du jeu de données fourni. Aucun recalcul périodique n'a été mis en place, puisque les données sources ne sont pas mises à jour régulièrement dans le cadre du projet.

Cette approche permet de conserver les traitements analytiques indépendants des consultations utilisateur. En cas d'évolution du jeu de données, la couche Gold pourra être reconstruite à partir des données Silver mises à jour.

4.2.3 Implémentation de l'API backend

Le backend applicatif a été développé en Python et déployé sous la forme d'une fonction AWS Lambda, accessible au travers d'Amazon API Gateway.

L'implémentation utilise une seule fonction Lambda pour les différents endpoints de consultation. Ce choix repose sur le périmètre limité du MVP, dans lequel les opérations partagent une logique similaire à savoir vérifier les paramètres reçus, interroger les résultats Gold avec Athena et construire une réponse au format JSON. Cette approche permet de mutualiser la logique d'accès aux données et de limiter le nombre de fonctions, de rôles IAM et de ressources à administrer. Elle constitue toutefois un compromis, puisque les endpoints partagent les mêmes ressources d'exécution et la même configuration. Une séparation en plusieurs fonctions pourrait être envisagée si leurs besoins évoluaient significativement.

La communication avec Athena est réalisée à l'aide du SDK AWS pour Python (`boto3`). Lorsqu'une requête nécessite des données analytiques, la Lambda soumet une requête SQL à Athena, attend son exécution, puis récupère les résultats et les convertit en objets JSON. L'API expose les fonctionnalités suivantes :

Endpoint	Fonction
`GET /api/v1/health`	Vérification du fonctionnement du backend
`GET /api/v1/filters`	Récupération des exportateurs, produits et fréquences disponibles
`GET /api/v1/heatmap`	Récupération des données agrégées pour la heatmap
`GET /api/v1/anomalies`	Consultation des anomalies selon les filtres
`GET /api/v1/anomalies/{id}`	Récupération des informations détaillées d'une anomalie

Ces différents endpoints ont été configurés dans Amazon API Gateway, comme illustré à la Figure X. Les routes de consultation sont associées à une même fonction Lambda, qui centralise la logique applicative et l'accès aux résultats analytiques. Les fonctionnalités de génération des rapports utilisent quant à elles des routes et une fonction Lambda distinctes, tout en partageant le même point d'entrée API Gateway.

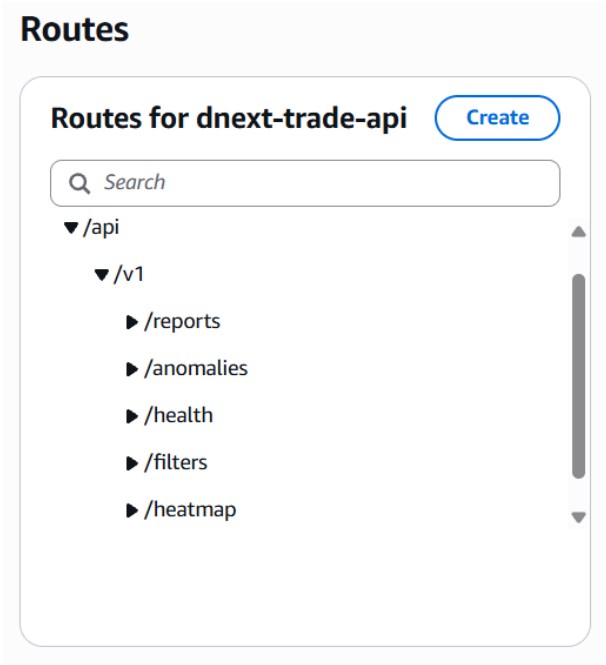


Figure 3: Configuration des routes HTTP du backend dans Amazon API Gateway

L'endpoint de la *heatmap* permet de sélectionner un pays exportateur, un produit et une fréquence mensuelle, trimestrielle ou annuelle. Il regroupe ensuite les résultats selon les périodes demandées et les dix principaux partenaires commerciaux, sélectionnés selon leur valeur commerciale. Chaque cellule contient notamment le nombre de flux observés, le nombre de correspondances valides et le nombre d'anomalies. Un statut est également retourné afin d'indiquer si la cellule contient une anomalie, des échanges cohérents ou uniquement des flux sans correspondance exploitable. Les endpoints consacrés aux anomalies permettent quant à eux de récupérer une liste de cas détectés, puis de consulter les informations détaillées d'un flux à partir de son identifiant.

Enfin, une validation des paramètres est effectuée avant l'exécution des requêtes SQL. Elle permet notamment de vérifier le format des codes pays, des codes HS4 et des périodes, ainsi que de retourner des erreurs HTTP adaptées lorsque les paramètres sont invalides.

4.2.4 Déploiement et intégration AWS

Le déploiement du backend a été réalisé dans la région AWS eu-north-1, à l'aide de templates AWS CloudFormation. Cette approche permet de définir les ressources sous forme de code (*Infrastructure as Code*) et de reproduire leur configuration sans devoir les créer manuellement depuis la console AWS.

Deux stacks indépendants ont été définies. La première, `dnext-trade-gold`, crée le bucket S3 destiné aux résultats analytiques. La seconde, `dnext-trade-backend`, regroupe la fonction Lambda, son rôle IAM, API Gateway et les autorisations nécessaires à leur communication.

La fonction Lambda utilise Python 3.12 et dispose de permissions lui permettant d'exécuter des requêtes Athena, de consulter les métadonnées du Glue Data Catalog et de lire les données Gold stockées dans S3. Ces permissions sont définies dans CloudFormation afin de limiter les accès aux ressources nécessaires à son fonctionnement.

Le déploiement est également facilité par deux scripts Bash. Le script `build_gold.sh` permet de créer, remplir et vérifier la couche Gold. Il évite notamment de recalculer les résultats lorsque la table est déjà remplie et propose une option permettant de reconstruire les données après une nouvelle ingestion. De son côté, `deploy_api.sh` automatise la préparation du code Python, son transfert vers S3 et le déploiement ou la mise à jour du backend.

L'API est exposée à travers un endpoint HTTPS fourni par API Gateway. Une configuration CORS permet à l'interface web, déployée indépendamment, d'effectuer des requêtes depuis un navigateur. La

même API Gateway est également partagée avec la composante de génération des rapports, dont les routes utilisent une fonction Lambda distincte.

Enfin, les endpoints ont été documentés au format OpenAPI afin de préciser les paramètres acceptés et les réponses attendues. Cette documentation constitue le contrat d'intégration entre le backend et les autres composants de la plateforme.

5 Tests et évaluation

Ce chapitre présente les résultats des tests effectués sur la plateforme et évalue dans quelle mesure l'implémentation répond aux exigences définies au chapitre 2. Il présente également les principales limites du MVP et les améliorations qui seraient nécessaires pour envisager une utilisation réelle au sein de DNEXT.

5.1 Validation des exigences fonctionnelles

Les principales fonctionnalités du système ont été évaluées par rapport aux exigences fonctionnelles définies initialement. Le tableau suivant présente leur état de validation.

Exigence	Évaluation
FR-01 : Ingestion des données	Satisfaite. Le jeu de données UN Comtrade de 200'587 déclarations a été ingéré et rendu accessible avec Athena.
FR-02 : Mirror matching	Satisfaite. Les déclarations correspondantes sont associées par Athena, tout en conservant les cas sans miroir.
FR-03 : Détection des anomalies	Satisfaite. Les prix unitaires FOB/CIF et les écarts sont calculés, avec 27'467 anomalies identifiées.
FR-04 : API documentée	Satisfaite. Les résultats sont accessibles à travers API Gateway et Lambda, avec une documentation OpenAPI.
FR-05 : Paramètres d'analyse	Disponible côté backend. L'API prend en charge les pays exportateurs, les produits et les fréquences temporelles. La validation finale avec l'interface reste à effectuer.
FR-06 : Heatmap des partenaires	Disponible côté backend. L'API fournit les données agrégées des dix principaux partenaires selon les filtres sélectionnés.
FR-07 : Sélection des anomalies	Disponible côté API grâce aux identifiants individuels. Le fonctionnement complet avec l'interface doit être confirmé.
FR-08 : Explication IA	Le workflow de génération et de stockage des rapports est implémenté. Les explications exploitent le contexte historique disponible, mais pas encore de sources externes.

Cette évaluation montre que les principales fonctionnalités analytiques et applicatives ont été implémentées. Les exigences liées à l'interface utilisateur et à la génération des rapports nécessitent toutefois une validation finale de l'intégration entre les différents composants.

Concernant les exigences non fonctionnelles, la séparation entre le pipeline de données, le backend et l'interface répond notamment aux objectifs de découplage et de déploiement indépendant (NFR-02 et NFR-03). Le recours aux services managés et serverless répond également à la volonté de limiter la maintenance et les ressources actives en permanence.

En revanche, la capacité de montée en charge (NFR-01) et la maîtrise des coûts à plus grande échelle (NFR-04) restent principalement justifiées par l'architecture retenue, sans avoir fait l'objet de tests approfondis.

5.2 Tests et résultats obtenus

La validation technique a été effectuée à plusieurs niveaux, en combinant des tests unitaires, des vérifications des données analytiques et des tests d'intégration sur AWS.

5.2.1 Validation des données et du mirror matching

Des requêtes SQL ont été exécutées dans Amazon Athena afin de vérifier les résultats du traitement analytique.

Les 200'587 déclarations initiales ont permis de produire 142'451 flux commerciaux distincts dans la couche Gold. Parmi ceux-ci, 57'355 disposent de déclarations correspondantes avec des prix calculables, dont 27'467 présentent une anomalie.

Les contrôles ont également permis de vérifier l'unicité des identifiants générés et de confirmer que les déclarations sans miroir, les valeurs non calculables et les partenaires agrégés sont conservés avec des statuts distincts.

Ces vérifications confirment la cohérence structurelle du traitement avec les règles définies. Elles ne constituent toutefois pas une validation économique individuelle de chaque anomalie, dont certaines peuvent provenir d'erreurs de déclaration ou de différences méthodologiques entre les pays.

5.2.2 Tests unitaires et d'intégration

Des tests unitaires ont été développés pour les principaux composants de la plateforme, afin de vérifier leur comportement indépendamment des services AWS.

Le pipeline d'ingestion dispose de 25 tests couvrant notamment la validation et la transformation des données. Le backend quant à lui comporte 7 tests portant sur la validation des paramètres, la gestion des erreurs et les opérations de consultation. Enfin, 24 tests concernent le workflow de génération des rapports IA, notamment la préparation du contexte, le traitement des réponses du modèle et la génération des documents.

Des réponses simulées (*mocks*) sont utilisées lorsque cela est nécessaire afin de tester les composants sans effectuer de véritables appels aux services AWS. Cette approche permet de vérifier rapidement la logique applicative et de limiter les coûts liés aux tests.

Au total, les 56 tests unitaires présents dans la version du dépôt final ont été exécutés localement avec succès. La Figure ci-dessous en guise d'exemple présente plus particulièrement le résultat des 7 tests du backend.

```
dnext-backend-mvp $ python3 -m unittest discover -s tests -v
test_anomalies_limit_validation (test_app.TestApi.test_anomalies_limit_validation) ... ok
test_anomaly_id_validation (test_app.TestApi.test_anomaly_id_validation) ... ok
test_health_does_not_call_athena (test_app.TestApi.test_health_does_not_call_athena) ... ok
test_heatmap_sql_and_status (test_app.TestApi.test_heatmap_sql_and_status) ... ok
test_validation_bad_month_blocked (test_app.TestApi.test_validation_bad_month_blocked) ... ok
test_validation_scope_required (test_app.TestApi.test_validation_scope_required) ... ok
test_validation_sql_injection_blocked (test_app.TestApi.test_validation_sql_injection_blocked) ... ok

----------------------------------------------------------------------
Ran 7 tests in 0.002s

OK
```

Figure 4: Résultat de l'exécution des tests unitaires du backend

Des tests d'intégration ont également été réalisés directement sur AWS afin de vérifier la communication entre les services déployés.

Les tests réalisés sur l'API déployée ont notamment permis de valider les endpoints de santé et de consultation de la *heatmap*, qui ont retourné des réponses HTTP 200. L'utilisation d'un code produit invalide a correctement entraîné une réponse HTTP 400.

Une requête OPTIONS a également retourné un statut HTTP 204, confirmant la prise en charge de la requête de prévalidation CORS. Ces vérifications constituent une première validation des échanges HTTP, sans remplacer des tests exhaustifs de l'ensemble des routes et des en-têtes de réponse.

```
~ $ printf "GET /health : "
GET /health : ~ $ curl -sS -o /dev/null -w "%{http_code}\n" \
>   "$API/api/v1/health"
200
~ $
~ $ printf "GET /heatmap : "
GET /heatmap : ~ $ curl -sS -o /dev/null -w "%{http_code}\n" \
>   "$API/api/v1/heatmap?exporter=ARG&product=1001&frequency=yearly"
200
~ $
~ $ printf "GET /heatmap (invalid product) : "
GET /heatmap (invalid product) : ~ $ curl -sS -o /dev/null -w "%{http_code}\n" \
>   "$API/api/v1/heatmap?exporter=ARG&product=INVALID"
400
~ $
~ $ printf "OPTIONS /heatmap (CORS) : "
OPTIONS /heatmap (CORS) : ~ $ curl -sS -o /dev/null -w "%{http_code}\n" \
>   -X OPTIONS "$API/api/v1/heatmap" \
>   -H "Origin: https://example.com" \
>   -H "Access-Control-Request-Method: GET"
204
~ $
```

Figure 5: Vérification des réponses HTTP du backend déployé sur AWS

Ces résultats confirment le bon fonctionnement des principales fonctionnalités de consultation. La génération des rapports IA dispose quant à elle d'un script de test de bout en bout permettant de déclencher un rapport, de suivre son statut et de télécharger le PDF produit. L'exécution réussie de ce scénario sur l'environnement AWS final reste à confirmer.

Les tests réalisés constituent ainsi une première validation technique du MVP, mais ne remplacent pas des essais approfondis de charge, de résilience et d'utilisation simultanée des différents composants.

5.3 Limites de la solution

Bien que la plateforme réponde aux principaux besoins du projet, plusieurs limites doivent être prises en compte avant d'envisager son utilisation dans un environnement réel.

5.3.1 Actualisation des données

La première limite concerne l'actualisation des résultats analytiques. Pour le MVP, l'ingestion repose sur le jeu de données UN Comtrade fourni, tandis que le calcul Gold est exécuté ponctuellement après la préparation des données.

Le système ne dispose donc pas encore d'un mécanisme permettant de reconstruire automatiquement les résultats lorsqu'une nouvelle version du jeu de données est disponible.

Cette approche reste adaptée au projet actuel, mais serait insuffisante pour DNEXT si de nouvelles données commerciales devaient être intégrées régulièrement.

Une évolution consisterait à déclencher automatiquement la reconstruction de la couche Gold après la validation d'une nouvelle ingestion, tout en conservant les anciens résultats disponibles jusqu'à la fin du traitement.

5.3.2 Performances et scalabilité

L'architecture serverless permet d'adapter les capacités de calcul à la demande, mais certains composants présentent des limites.

Les fonctions AWS Lambda standards possèdent notamment une durée maximale d'exécution de 15 minutes. Cette contrainte pourrait devenir problématique pour des traitements d'ingestion beaucoup

plus volumineux, particulièrement lorsque les fichiers doivent être entièrement chargés et transformés en mémoire.

Le calcul de *mirror matching* est quant à lui exécuté par Athena et n'est donc pas soumis directement à cette limite Lambda. Son coût et son temps d'exécution dépendraient toutefois du volume de données à traiter.

Concernant le backend, les requêtes Athena sont actuellement exécutées de manière synchrone depuis AWS Lambda. La fonction attend les résultats de la requête avant de retourner une réponse HTTP au frontend.

Dans l'implémentation actuelle, la Lambda dispose d'un délai maximal d'exécution de 29 secondes, tandis que l'attente des résultats Athena est limitée à environ 20 secondes. Sa concurrence est également limitée à deux exécutions simultanées afin de maîtriser la consommation des ressources.

Ces paramètres sont adaptés à une utilisation ponctuelle du MVP, mais pourraient devenir contraignants si le nombre de consultations augmentait significativement. Une hausse du trafic pourrait notamment entraîner des requêtes refusées ou des délais d'attente plus importants.

Pour une utilisation réelle, il serait donc nécessaire de réaliser des tests de charge afin d'évaluer la latence des requêtes et d'adapter les limites de concurrence. Selon les résultats, des mécanismes de cache ou une optimisation des requêtes Athena pourraient également être envisagés.

5.3.3 Qualité des données et détection des anomalies

La méthode de détection repose sur un seuil fixe de 35 % ainsi que sur la comparaison entre les prix FOB et CIF. Elle permet d'identifier des incohérences potentielles, mais ne suffit pas à déterminer leur origine.

Les différences observées peuvent notamment provenir des conditions de transport, de différences de quantités déclarées, de décalages temporels ou d'erreurs dans les données sources.

De plus, une part importante des observations ne possède pas de déclaration miroir exploitable. Ces situations sont correctement identifiées par le traitement, mais elles limitent le nombre de flux pour lesquels une comparaison peut effectivement être réalisée.

Une validation des règles de détection avec les experts métier de DNEXT serait donc nécessaire avant d'utiliser ces résultats dans un contexte décisionnel réel.

Par ailleurs, le projet se limite actuellement aux déclarations commerciales fournies par UN Comtrade, sans intégrer de jeu de données externe consacré aux coûts de transport. Le seuil de détection est également fixé à 35 %, sans possibilité de modification depuis l'interface. L'intégration de données de fret et la configuration des seuils selon les produits ou les marchés constitueraient donc des améliorations pertinentes pour DNEXT.

5.3.4 Limites des explications générées par IA

Le workflow de génération des rapports utilise les informations de l'anomalie sélectionnée ainsi que des données historiques et des comparaisons avec d'autres partenaires commerciaux, récupérées depuis la couche Gold à l'aide d'Athena. Ces informations fournissent au modèle un premier contexte quantitatif permettant de décrire l'évolution des échanges et de mettre en évidence certaines différences entre partenaires. En revanche, l'implémentation actuelle ne récupère pas automatiquement de sources externes, telles que les tarifs douaniers, les sanctions économiques, les conflits ou les événements susceptibles d'influencer les prix commerciaux. Le modèle peut donc proposer des hypothèses explicatives à partir des informations disponibles, mais ne dispose pas nécessairement des éléments permettant de vérifier leur validité.

Cette limite est particulièrement importante pour DNEXT, puisque la détection d'un écart statistique ne suffit pas à en identifier la cause économique. Les rapports générés doivent donc être considérés

comme une aide à l'analyse plutôt que comme une explication définitive. L'intégration de sources externes fiables et vérifiables représenterait ainsi une amélioration importante pour une utilisation professionnelle de la plateforme.

5.3.5 Sécurité et exploitation

Enfin, l'API est actuellement accessible sans authentification, afin de faciliter les tests et la démonstration du projet. Cette configuration ne serait pas suffisante pour une plateforme destinée à des utilisateurs professionnels. Un contrôle d'accès, une gestion plus stricte des domaines autorisés et une surveillance des erreurs et des coûts seraient nécessaires avant sa mise en production.

5.4 Perspectives d'évolution pour DNEXT

Dans le cadre d'un déploiement réel chez DNEXT, les améliorations devraient prioritairement porter sur l'automatisation des traitements, la robustesse des services et l'enrichissement des analyses.

- La première évolution consisterait à automatiser la mise à jour de la chaîne de données. L'arrivée de nouvelles données pourrait déclencher leur préparation, puis le recalcul des résultats Gold, avec un mécanisme de validation avant leur publication.
- Une seconde amélioration concernerait la montée en charge. Des tests de performance permettraient de déterminer si Athena et Lambda restent adaptés à une augmentation du volume de données et du nombre d'utilisateurs. Selon les résultats, des optimisations des requêtes, des mécanismes de cache ou une séparation de certains traitements pourraient être envisagés, sans introduire prématurément une infrastructure plus complexe.
- L'enrichissement des rapports IA représenterait également une évolution importante. L'intégration de sources externes fiables, notamment sur les coûts de transport, les politiques commerciales et les événements géopolitiques, permettrait de contextualiser davantage les anomalies. Les explications devraient toutefois conserver leurs références et pouvoir être vérifiées par un analyste.
- Enfin, la sécurisation de l'API, la supervision des services et le suivi des coûts devraient être renforcés pour permettre une exploitation durable de la plateforme.

Ces évolutions permettraient de faire évoluer le MVP vers une solution plus robuste, tout en conservant les principes de découplage et d'utilisation de services managés retenus lors de la conception.

6 Conclusion

Ce projet a permis de concevoir et d'implémenter une plateforme cloud-native destinée à l'analyse des données du commerce international et à la détection d'anomalies entre les déclarations d'importation et d'exportation. L'architecture retenue repose principalement sur des services managés AWS, permettant de séparer l'ingestion des données, les traitements analytiques, le backend et l'interface utilisateur.

L'implémentation du *mirror matching* avec Amazon Athena a notamment permis de traiter les 200'587 déclarations fournies et d'identifier 27'467 anomalies potentielles. Les résultats précalculés sont accessibles au travers d'une API, tandis que l'interface web et la composante d'intelligence artificielle permettent respectivement de visualiser les anomalies et de produire des rapports explicatifs. Les tests réalisés ont également permis de valider le fonctionnement des principaux composants de la plateforme.

Néanmoins, comme discuté dans la section précédente, plusieurs améliorations restent nécessaires avant d'envisager une utilisation réelle chez DNEXT, notamment l'automatisation des mises à jour des données, le renforcement de la sécurité et l'intégration de sources de contexte externes pour enrichir les explications générées par IA.

Au final, le projet démontre concrètement la faisabilité d'une telle plateforme avec une architecture serverless relativement simple et adaptée au volume actuel de données. Il constitue ainsi une première base fonctionnelle pouvant évoluer progressivement selon les besoins de DNEXT, sans nécessiter dès le départ une infrastructure plus complexe.

7 Tables des figures

Figure 1: Architecture générale AWS retenue....18
Figure 2: Répartition des flux commerciaux selon leur statut de correspondance et nombre d'anomalies détectées....20
Figure 3: Configuration des routes HTTP du backend dans Amazon API Gateway 22
Figure 4: Résultat de l'exécution des tests unitaires du backend....25
Figure 5: Vérification des réponses HTTP du backend déployé sur AWS....26

8 Bibliography

[1] AWS, «Use columnar storage formats,» 2026. [En ligne]. Available: https://docs.aws.amazon.com/athena/latest/ug/columnar-storage.html. [Accès le 2026].

[2] AWS, «Create databases and tables,» 2026. [En ligne]. Available: https://docs.aws.amazon.com/athena/latest/ug/work-with-data.html. [Accès le 2026].

[3] «Redshift vs Athena,» 2021. [En ligne]. Available: https://www.reddit.com/r/aws/comments/umx793/redshift_vs_athena/?captcha=1. [Accès le 2026].

[4] N. Hallett, «Redshift vs. Athena vs. EMR – AWS' big data solutions explained,» 9 8 2023. [En ligne]. Available: https://www.justaftermidnight247.com/insights/redshift-vs-athena-vs-emr-aws-big-data-solutions-explained/. [Accès le 2026].

[5] AWS, «What is AWS Lambda,» 2026. [En ligne]. Available: https://docs.aws.amazon.com/lambda/latest/dg/welcome.html. [Accès le 2026].

[6] Dash0, «AWS ECS vs. EKS: 5 Key Differences and How to Choose,» 24 2 2026. [En ligne]. Available: https://www.dash0.com/comparisons/ecs-vs-eks-5-key-differences-and-how-to-choose. [Accès le 2026].

[7] L.-O. Amir, «AWS Fargate explained: Pros and cons, components & key features,» 21 7 2023. [En ligne]. Available: https://www.flexera.com/blog/finops/aws-fargate-explained-pros-cons-components-features/. [Accès le 2026].

[8] AWS, «Create a table from query results (CTAS),» 2026. [En ligne]. Available: https://docs.aws.amazon.com/athena/latest/ug/ctas.html. [Accès le 2026].

[9] AWS, «Get started with a CloudFront standard distribution,» 2026. [En ligne]. Available: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/GettingStarted.SimpleDistribution.html. [Accès le 2026].

[10] AWS, «Welcome to AWS Amplify Hosting,» 2026. [En ligne]. Available: https://docs.aws.amazon.com/amplify/latest/userguide/welcome.html. [Accès le 2026].

[11] AWS, «Amazon Bedrock pricing,» 2026. [En ligne]. Available: https://aws.amazon.com/bedrock/pricing/?upgrade-intent=undefined. [Accès le 2026].

[12] AWS, «What is Step Functions?,» 2026. [En ligne]. Available: https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html. [Accès le 2026].

[13] AWS, «Amazon Bedrock Overview,» 2026. [En ligne]. Available: https://docs.aws.amazon.com/en_en/bedrock/latest/userguide/what-is-bedrock.html. [Accès le 2026].